



Full-stack software development

Front-end: Internationalisation/i18n

R. Naudts, J. Pieck, J. Rombouts, B. Van Impe

Internationalisation/i18n

- Often abbreviated as **i18n**, where 18 stands for the number of letters between “i” and “n”.
- Refers to the process of designing a software application so that it can be adapted to various languages and regions without code changes.
- Make software universally accessible and usable, regardless of language and region.

note: *don't confuse this with l10n (localisation):*

This is the process of adapting a product to a specific market. This includes changes in date formats, weights, dealing with difference between 2 variants of the same language, etc.

Key aspects

- **Separate content from code**

- Text is stored separately from source code, in resource files. This allows easy translation, without altering the code base.

- **Locale aware**

- Locale is a set of parameters that represent a users' language and country (e.g. en_US)
- Software can behave differently based on the locale, by supporting multiple languages.



Welcome!

Courses lets you see as a lecturer all the courses you are teaching and as a student all the courses you are enrolled in. You can also see when the courses are scheduled and the students enrolled in each course.



Bienvenido!

Cursos le permite ver como profesor todos los cursos que está impartiendo y como alumno todos los cursos en los que está matriculado. También puede ver cuándo están programados los cursos y los alumnos matriculados en cada curso.

Login

Login succesful. Redirecting to homepage...

Username:

Password:

[Login](#)

Inicio de sesión

Inicio de sesión exitoso. Redirigiendo a la página de inicio...

Nombre de usuario:

Contraseña:

[Conéctese](#)

Next-intl

- To implement i18n in the Next.js App Router, we will use a modern library called **next-intl**.
 - Handles translations for server components: it prepares translations on the server.
 - Handles translations for client components: it provides a useTranslations hook.
- <https://next-intl.dev/docs/environments/server-client-components>

Next-intl

The 5 steps for working with next-intl:

1. **Middleware:** The “traffic controller” that reads the URL (/en or /es) and determines the language.
2. **JSON Translation Files:** The “dictionaries” where our keys and translations are stored.
3. **Config Loader:** The “glue” that knows which “dictionary” to load based on the language.
4. **Provider:** The “box” (just like AuthContext) that makes the loaded translations available to the whole app/layout.tsx].
5. **Hooks:** The “tools” to actually get the translations in our components.

Middleware

- **/middleware.ts** is a file that runs for every request.
- The createMiddleware function reads the URL, determines the locale (e.g., “en”), and sets the language for the request.

```
import createMiddleware from 'next-intl/middleware';
```

```
export default createMiddleware({  
  locales: ['en', 'es'],  
  defaultLocale: 'en',  
});
```

```
export const config = {
```

```
  matcher: ['/(?!api|_next/static|_next/image|favicon.ico).*/'],  
};
```

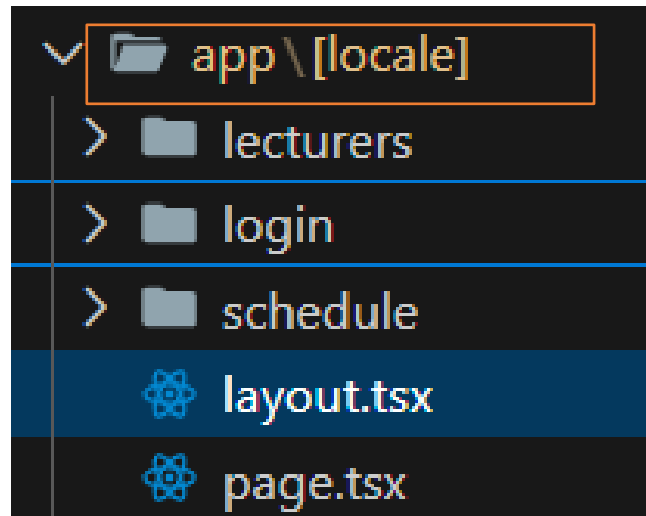
→ The languages that we support

→ The fallback language

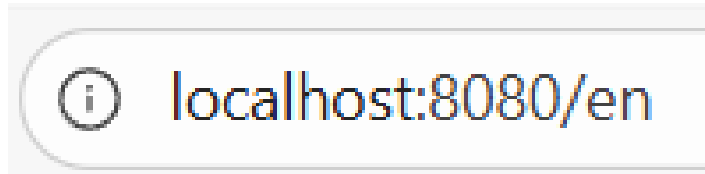
→ A regular expression that specifies to only run the middleware on actual page requests

Middleware

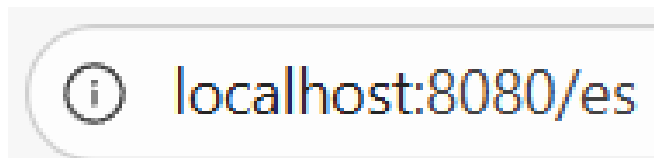
- For the middleware to be able to read the language from the url, we need to add it to all routes.
- It's a dynamic part of all routes in the app router:



The default starting URL is now:

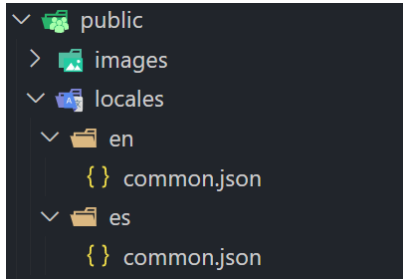


When we switch to Spanish (see later):



The translation files

- For each language, we create a JSON file in public/locales/. We structure the translations into “scopes” (groups) that often match our components.



```
{
  "app.title": "Courses App",
  "general.error": "An error has occurred. Please try again later.",
  "header": {
    "welcome": "Welcome",
    "nav": {
      "home": "Home",
      "lecturers": "Lecturers",
      "schedules": "Schedules",
      "login": "Login",
      "logout": "Logout"
    }
  },
  "login": {
    "title": "Login",
    "button": "Login",
    "success": "Login succesful. Redirecting to homepage...",
    "validate": { ...
  },
  "label": { "username": "Username:", "password": "Password:" }
},
"home": { ...
}
}
```

public/locales/en/common.json

```
{
  "app.title": "Cursos App",
  "general.error": "Se ha producido un error. Vuelva a intentarlo más tarde.",
  "header": {
    "welcome": "Bienvenido",
    "nav": {
      "home": "Inicio",
      "lecturers": "Conferenciantes",
      "schedules": "Horarios",
      "login": "Conéctese",
      "logout": "Cerrar sesión"
    }
  },
  "login": {
    "title": "Inicio de sesión",
    "button": "Conéctese",
    "success": "Inicio de sesión exitoso. Redirigiendo a la página de inicio...",
    "validate": { ...
  },
  "label": { "username": "Nombre de usuario:", "password": "Contraseña:" }
},
"home": { ...
}
}
```

public/locales/es/common.json

The config loader

- **/i18n.ts**: This file (running on the server) receives the locale (e.g., “es”) from the middleware. It then dynamically loads the correct JSON file from the public folder.
 - If the request return undefined, we set it to an empty string and next-intl will default to the defaultLocale we set in the middleware.

```
import { getRequestConfig } from "next-intl/server";

export default getRequestConfig(async ({ requestLocale }) => {
  const locale = (await requestLocale) || "";

  return {
    messages: (await import(`./public/locales/${locale}/common.json`)).default,
    locale,
  };
});
```

next.config.js

- Before our middleware can work, we must install **next-intl** as a plugin. This file is the first thing Next.js reads when you start your development server (with **npm run dev**).

```
const withNextIntl = require('next-intl/plugin')('./i18n.ts');  
  
/** @type {import('next').NextConfig} */  
const nextConfig = {};  
  
module.exports = withNextIntl(nextConfig);
```

/next.config.js

- What's happening here?
 - withNextIntl(nextConfig) injects next-intl's logic deep into the Next.js development server.

next.config.js

- Why next.config.**js** and not next.config.**ts**?
 - This is the **very first** file Next.js reads when you type npm run dev.
 - At that initial startup moment, the TypeScript compiler **has not run yet**.
 - Next.js (and the underlying Node.js environment) expects a standard **JavaScript file** (using require and module.exports) to configure the development server *itself*.

NextIntlClientProvider

- Just like AuthProvider, we must “provide” the translations to our component tree.
- We do this in the root layout.tsx using the <NextIntlClientProvider>.
- Provide the messages, which are read server-side, to **client** components.
- Server components do not read from this provider. They fetch their translations directly.

NextIntlClientProvider

```
const RootLayout = async ({
  children,
  params,
}): {
  children: React.ReactNode;
  params: Promise<{ locale: string }>;
}) => {
  const { locale } = await params;
  const messages = await getMessages();

  return (
    <html lang={locale}>
      <body>
        <NextIntlClientProvider locale={locale} messages={messages}>
          <AuthProvider>
            <Header />
            <main className="...">
              {children}
            </main>
          </AuthProvider>
        </NextIntlClientProvider>
      </body>
    </html>
  );
};

export default RootLayout;
```

Await the messages from i18n.ts

Wrap the app in the Provider and pass the messages

Use translations

- Now that the Provider is set up, we can use translations.
 - In server components (e.g., page.tsx):
 - These components are async.
 - We use: `await getTranslations()` (from `next-intl/server`)
 - In client components (e.g., `UserLoginForm.tsx`):
 - These components have 'use client'.
 - We use: `useTranslations()` (from `next-intl`)

Server components

```
import Image from 'next/image';

import { getTranslations } from 'next-intl/server';
import coursesLogo from '@assets/images/courses.png';

const Home = async () => {

  const t = await getTranslations('Home');

  return (
    <>
      <span>

        <h1>{t('title')}</h1>
      </span>
      <div>
        <p>{t('description')}</p>
      </div>
    </>
  );
};

export default Home;
```

Reads the config in i18n.ts, gets the messages directly out of common.json with the specified key.

The t(<key>) function finds the translation for the corresponding key in the messages.

Client components

```
'use client';
import { useTranslations } from 'next-intl';

export const UserLoginForm = () => {
  const t = useTranslations('UserLoginForm');

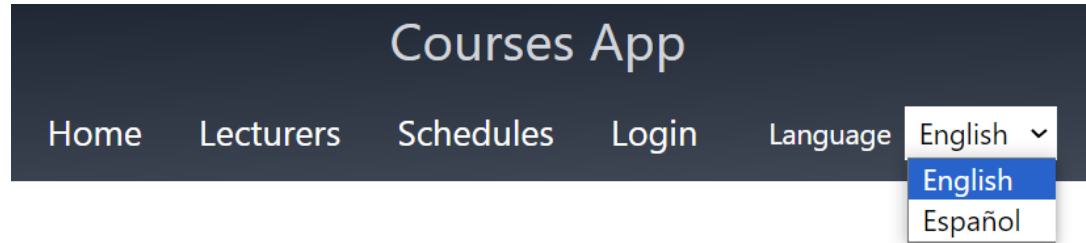
  return (
    <form onSubmit={handleSubmit}>
      {/* 3. Use the 't' function */}
      <h2>{t('title')}</h2>

      <label htmlFor="username">{t('username')}</label>
      <button type="submit">{t('loginButton')}</button>
    </form>
  );
};
```

In client components, we must use the `useTranslations` hook. It reads the messages provided by `NextIntlClientProvider`.

The `t(<key>)` function finds the translation for the corresponding key in the messages.

Language component



```
const Language: React.FC = () => {
  const router = useRouter();
  const { locale, pathname, asPath, query } = router;

  const handleLanguageChange = (event: { target: { value: string } }) => {
    const newLocale = event.target.value;
    const { pathname, asPath, query } = router;
    router.push({ pathname, query }, asPath, { locale: newLocale });
  };

  return (
    <div className="ml-6">
      <label htmlFor="language" className="text-white">
        Language
      </label>
      <select
        id="language"
        className="ml-2 p-1"
        value={locale}
        onChange={handleLanguageChange}>
        <option value="en">English</option>
        <option value="es">Español</option>
      </select>
    </div>
  );
};
```